

新手实操全链路：通过 CANN-Learning-Hub 入门，CANNBot 快速完成首个算子开发

CANN核心贡献者—谢好冉



目录 CONTENTS

01

开场：算子开发的难点

解析算子在AI计算中的核心价值，深度剖析新手入门时面临的环境搭建、调试困难、资料匮乏“三重门”困境。

02

CANN-Learning-Hub 新手灯塔

介绍体系化的学习资源平台，通过结构化教程、实战案例与工具指引，为开发者照亮前行方向，大幅降低学习门槛。

03

CANNBot 重塑开发的AI智能体

揭秘革命性AI工具CANNBot，它如何将算子开发从“数天”压缩至“分钟”级，实现自动化编码与优化的质的飞跃。

04

实战：CANNBot助力实现GeluGradV2算子

通过真实模拟新手视角，完整演示使用CANNBot从需求分析到算子生成、验证的全过程，直观感受效率提升。

01 新手入门的“三重门”：高门槛与长周期



第一重门：环境配置难

环境配置困难，需要安装固件、驱动、软件包、以及相关依赖



第二重门：知识体系复杂

学习曲线陡峭，需啃下海量硬件手册，理解Tiling策略、数据搬移等晦涩概念，对算法工程师是极大挑战。



第三重门：开发过程缓慢

传统开发流程冗长，涵盖手写Ascend C代码、调试过程各种类型错误，环环相扣且容错率低，极易出现流程性失误。

02 CANN learning hub

before

- 1. Ascend C学习资源分布散
- 2. 官方文档阅读难、耗时长
- 3. 缺乏系统讲解
- 4. 深度不够，缺乏代码实战



after

- 1. CANN learning hub将算子开发知识集中起来，新手只需要看它就够了，某个API使用等零散知识可以再搜文档
- 2. 从基础知识开始讲解，降低了新手入门的难度

02 CANN learning hub

⚙️ 算子开发			
10 分钟体验自定义算子	快速上手：第一个自定义算子	在线体验	✅ 已上线
10 分钟体验算子 API 调用	快速体验调用 CANN 内置算子 API 进行计算	在线体验	✅ 已上线
Ascend C 算子开发系列（初级）：第一章 / 第二章	算子基础 / CANN 架构 / 算子开发基础 / Add 算子实践	在线体验	✅ 已上线
Ascend C 算子开发系列（中级）：第三章 / 第四章 / 第五章 / 第六章	Vector 算子开发 / Cube 算子开发 / 融合算子开发 / 开源算子仓	在线体验	✅ 已上线
Ascend C 算子开发系列（高级）：第七章 / 第八章	Ascend C 算子功能调试 / 性能优化	在线体验	✅ 已上线
算子开发实战系列	MC2 融合算子实战	在线体验	✅ 已上线
HiXL 单边通信开发	HiXL 应用开发教程	在线体验	✅ 已上线
算子开发练习	开放题库 / 历届算子赛真题等	CANNJudge 开放题库	✅ 已开放
参加算子赛	算子天梯赛 / 校园赛等各种大赛等你挑战	CANN 大赛专区	✅ 已开放

开发者视角亮点

- 1.增加算子基础，这是之前的资料里面很少有的，之前需要自己去了解
- 2.第三章增加专门的tiling策略讲解，帮助开发者理解复杂的数据搬运策略，包括核间切分，核内切分
- 3.增加cv融合和vv融合讲解，将融合算子讲透，这是之前官方文档和其他平台资源所没有涉及的。

传统开发的核心痛点总结

一个简单加法算子的平均开发周期：

3 - 5 天

01 算子分析难

自己分析设计算子时，算子精度不达标，复杂的算子需要考虑计算过程中精度丢失问题，要自己优化计算过程

痛点：精度低

02 代码编写慢

需要从0开始编写代码，枯燥，效率低，耗时长，有很多细节容易被忽略

痛点：开发效率低

03 新手调错难

日志阅读困难，排错难，主要依赖开发者经验

痛点：排错难

03 CANNbot: AI助力算子开发

CANNBot可以帮助新手开发者轻松上手算子开发，对于语法使用问题不用再查官方文档，不用自己根据复杂的计算公式手动堆积API，直接把计算公式发给CANNBot就可以。

before

- 1.算子开发问题频繁，很多细节新手注意不到
- 2.算子调错难，日志看不懂，只能发帖询问，解决耗时长
- 3.新手开发对流程不熟悉，算子开发交付困难
- 4.算子泛化功能实现困难，需要支持各种不同的case

after

- 1.CANNBot生成的代码问题少，语法问题少
- 2.报错信息直接发给CANNBot，它会提供解决方案
- 3.CANNBot支持算子开发全流程的生成与测试，可以生成相关的测试样例及本地进行门禁测试
- 4.CANNBot自动生成多个场景的测试case，不需要手写

04 实战：利用CANNBot自动化开发GeluGradV2算子

从需求梳理、智能生成到最终交付的全流程自动化解析，释放算子开发核心生产力

01 需求分析

核心目标：明确GeluGradV2算子的数学原型与核心功能，精准定义算子的输入输出维度、数据类型约束及关键性能指标，为自动化开发奠定基础。

02 提交需求

智能对接：将算子需求文档或自然语言描述直接提交给CANNBot智能体，利用大模型能力自动解析需求意图，提取关键开发要素。

03 代码生成

自动化编码：CANNBot自动生成符合CANN规范的Tiling并行策略、底层Kernel计算代码，以及配套的编译构建脚本，减少手工编码工作量。

04 迭代优化

性能调优闭环：基于自动生成的性能分析报告，智能体辅助定位瓶颈，迭代优化Tiling参数与Kernel逻辑，持续提升算子的执行效率与资源利用率。

05 完整交付

全量资产交付：自动产出标准化的README说明文档、覆盖核心场景的单元测试(UT)用例，以及可直接运行的算子调用示例(Example)，降低使用门槛。

总结：大幅提高算子开发效率，降低开发难度

CANNBot 生成效果展示：

我已经按照您的要求，将tanh模式的公式用AscendC API方式完全实现了：

对于approximate='tanh'模式，我使用了以下AscendC API调用来实现您提供的公式：

1. 公式分解为AscendC API调用：

```
- AscendC::Mul(exp_f_x, x, static_cast<float>(-2.0), this->processDataNum) - 计算-2*x
- AscendC::Exp(exp_f_x, exp_f_x, this->processDataNum) - 计算exp(-2*x)
- AscendC::Add(one_plus_exp_f_x, exp_f_x, static_cast<float>(1.0), this->processDataNum) - 计算1+exp(-2*x)
- AscendC::Div(t, static_cast<float>(1.0), one_plus_exp_f_x, this->processDataNum) - 计算t = 1/(1+exp(-2*x))
- AscendC::Mul(res_p, exp_f_x, t, this->processDataNum) - 计算res_p = exp(-2*x) * t (当x>0时)
- AscendC::Sub(one_minus_t, static_cast<float>(1.0), t, this->processDataNum) - 计算1-t
- AscendC::Add(res_p, static_cast<float>(1.0), one_minus_t, this->processDataNum) - 计算res_p = 1-(1-t) = t (当x<=0时)
- AscendC::Mul(temp, x, static_cast<float>(2.0), this->processDataNum) - 计算2*x
- AscendC::Mul(temp, temp, res_p, this->processDataNum) - 计算2xres_p
- AscendC::Add(gelu_grad, static_cast<float>(1.0), temp, this->processDataNum) - 计算1+2xres_p
- AscendC::Mul(gelu_grad, gelu_grad, t, this->processDataNum) - 计算最终结果
```

所有API调用都严格按照您提供的AscendC语法格式，没有使用任何C++运算符，完全符合您的要求。

在CANNBot的辅助下，缩短API堆积时间，不需要自己一步步拆分计算公式，不需要自己写设计文档，不用自己写测试样例。



全流程

算子核心代码、算子
交付件生成



生成质量高

代码质量较高，可以
直接用



低门槛

新手开发者不需要掌
握复杂的算子知识



更新迭代太快

喜欢尝鲜的可以及时更新最新
版本，求稳定可以选一个合适
的版本一直使用

CANNBot



CANN learning hub



THANK S

